

What is DATA and What Is SQL.

- What is Database and types of Database
- What is DBMS and SQL server.

What is Data

Data = Raw facts, figures, or observations that by themselves may not have much meaning. It can be **numbers, text, images, audio, video, logs, sensor readings, transactions**, etc.

Example

- 25 → Just a number (data).
- 25°C → Temperature in Celsius (now more meaningful).
- 25°C at 9:00 AM in Hyderabad → Structured and useful (information).

Types of Data

1. Structured Data

- o Organized in rows & columns (tables).
- o Example: Databases, Excel sheets, SQL tables.

OrderID	CustomerID	Product	Amount	Date
5001	101	Laptop	55000	2025-09-01
5002	102	Headphones	2000	2025-09-02

2. Semi-structured Data

- **Definition:** Has some structure (tags, key-value pairs) but not fixed like a table.
- **Storage:** JSON, XML, NoSQL databases (Cosmos DB, MongoDB).
- **Examples:**
 - o JSON files from APIs
 - o IoT sensor readings (temperature, location)
 - o Log files (timestamp, event, message)

Sample (JSON)

```
{
  "OrderID": 1001,
  "Customer": "Pawan",
  "Items": [
    {"Product": "Laptop", "Price": 55000},
    {"Product": "Mouse", "Price": 500}
  ],
  "Date": "2025-09-01"
}
```

3. Unstructured Data

- **Definition:** No predefined schema, hard to store in rows/columns.
- **Storage:** Data Lakes, Blob Storage, File Systems.
- **Examples:**
 - o Images, videos, audio files
 - o PDF documents, Word files
 - o Emails, social media posts, call recordings

Data vs. Information

- **Data** = Raw facts.
- **Information** = Processed data that has meaning and context.

Example:

- Data: "120, 130, 140"
- Information: "These are the blood pressure readings of a patient."

Types of data formats

1. Structured Data Formats

(Data stored in rows & columns → easy for databases, SQL)

- **CSV (Comma-Separated Values)** → Simple text files with tabular data.
- **TSV (Tab-Separated Values)** → Like CSV, but tab-separated.
- **XLS/XLSX** → Excel spreadsheet formats.
- **SQL Tables** → Stored in relational databases.

2. Semi-Structured Data Formats

(Data has some structure but not fixed → flexible, good for NoSQL & big data)

- **JSON (JavaScript Object Notation)** → Key-value pairs, widely used in APIs.
- **XML (Extensible Markup Language)** → Tag-based, often used in configs and web services.
- **YAML** → Human-readable, often used in configs (like Kubernetes, CI/CD).
- **Avro** → Compact, row-based, supports schema evolution (used in Kafka, Hadoop).

3. Unstructured Data Formats

(Data without a predefined structure → media, logs, free text)

- **Text files (.txt, .log)**
- **Images (.jpg, .png, .gif)**
- **Audio (.mp3, .wav)**
- **Video (.mp4, .avi, .mov)**
- **PDF / Word Documents**

4. Columnar Data Formats (Optimized for analytics & big data)

- **Parquet** → Columnar, highly efficient for queries (used in Spark, Hive).
- **ORC (Optimized Row Columnar)** → Similar to Parquet, used in Hadoop & Hive.
- **Delta** → Built on Parquet, supports ACID transactions (used in Databricks).

5. Binary Data Formats

- **Protocol Buffers (Protobuf)** → Compact, efficient for serialization.
- **MessagePack** → JSON-like but binary & smaller in size.
- **Feather / Arrow** → Fast data exchange in analytics & ML pipelines.

In short:

- **Structured** → CSV, SQL, Excel
- **Semi-structured** → JSON, XML, Avro
- **Unstructured** → Images, text, video
- **Columnar** → Parquet, ORC, Delta
- **Binary/Optimized** → Protobuf, Arrow

What is a Database

A **database** is an organized collection of **data** that can be easily accessed, managed, and updated. Databases store data in such a way that it can be retrieved efficiently, typically using a **Database Management System (DBMS)**.

Key Features of a Database:

- Efficient data storage and retrieval
- Supports CRUD operations (Create, Read, Update, Delete)
- Secures and backs up data
- Supports concurrent access by multiple users

Types of Databases (with Examples):

1. Relational Database (RDBMS)

- Stores data in **tables** (rows and columns)
- Uses **SQL** for querying
- **Data is structured**

Examples:

- Microsoft SQL Server
- MySQL
- PostgreSQL
- Oracle Database
- Azure SQL Database

Use case: Banking systems, enterprise applications, CRM

2. NoSQL Database

A **Non-Relational Database** stores data in a **flexible format** — not tables. It can store **structured, semi-structured, or unstructured** data.

Types of NoSQL:

Type	Description	Example
Document	JSON-like data	MongoDB, CouchDB
Key-Value	Simple key-value pairs	Redis, DynamoDB
Column-Family y	Columnar storage	Cassandra, HBase

Graph	Nodes and relationships	Neo4j, ArangoDB
--------------	-------------------------	-----------------

Use case: Real-time analytics, social networks, IoT, content management

3. Distributed Database

- Data is stored across multiple physical locations (servers or regions)
- Supports **horizontal scaling**

Examples:

- Google Spanner
- Amazon Aurora
- Azure Cosmos DB

Use case: Global applications, fault-tolerant systems

4. Cloud Database

- Hosted on cloud platforms like Azure, AWS, or GCP
- Managed and scalable

Examples:

- Azure SQL Database
- Amazon RDS
- Google Cloud Spanner

Use case: Modern cloud apps, SaaS platforms

5. Object-Oriented Database

- Stores data as **objects**, similar to object-oriented programming
- Supports complex data models

Examples:

- db4o
- ObjectDB

Use case: CAD/CAM, multimedia applications

6. Graph Database

- Focuses on **relationships** using nodes and edges
- Used for deep relationship queries

Examples:

- Neo4j
- Amazon Neptune

Use case: Fraud detection, recommendation systems, social networks

7. Time-Series Database

- Designed to store **timestamped data**

Examples:

- InfluxDB
- TimescaleDB
- Prometheus

Use case: IoT data, financial data, server monitoring

Summary Table:

Type	Structured?	Query Language	Best For
Relational	Yes	SQL	Structured, transactional data
NoSQL	No	Varies	Unstructured or fast-changing data
Distributed	Varies	Varies	Global-scale applications
Cloud	Varies	Varies	Scalable, managed hosting
Object-Oriented	Yes	OOP-style	Complex applications
Graph	No	Cypher (Neo4j)	Relationship-heavy queries
Time-Series	Yes	TS-optimized	Time-based data trends

what is SQL server

SQL Server is a **relational database management system (RDBMS)** developed by **Microsoft**.

It is software used to **store, organize, and manage data** in a structured way, and it allows users to interact with the data using **SQL (Structured Query Language)**.

Key Features of SQL Server:

1. **Relational Database** – Data is stored in **tables** (rows & columns).
2. **SQL Support** – Uses T-SQL (Transact-SQL), Microsoft's version of SQL, for queries.
3. **Secure & Scalable** – Can handle small applications to very large enterprise-level systems.
4. **Tools** – Comes with **SQL Server Management Studio (SSMS)** for easy database design, queries, and management.
5. **Integration** – Works well with other Microsoft services like **Azure, Power BI, Excel, .NET apps**.
6. **High Availability** – Supports **backup, replication, failover clustering** to avoid downtime.

Example Use Cases:

- **Banking systems**: Storing transactions and customer accounts.
- **E-commerce websites**: Managing products, orders, and customers.
- **Healthcare**: Storing patient records securely.
- **Business Intelligence (BI)**: Data warehousing and reporting with tools like Power BI.

Types of SQL Commands

1. DDL (Data Definition Language)

Used to **define or modify** database structure (tables, schemas, etc.).

- CREATE → create tables, databases, views.
- ALTER → modify table structure (add/remove columns).
- DROP → delete tables or databases.
- TRUNCATE → delete all rows in a table (but keep structure).

2. DML (Data Manipulation Language)

Used to **insert, update, delete, or retrieve** data from tables.

- INSERT → add new records.
- UPDATE → modify existing records.
- DELETE → remove records.
- SELECT → fetch data (sometimes grouped under DQL).

3. DQL (Data Query Language)

Some books separate this from DML. It mainly includes:

- SELECT → used to query data.

4. DCL (Data Control Language)

Deals with **permissions and security** of the database.

- GRANT → give access rights.
- REVOKE → remove access rights.

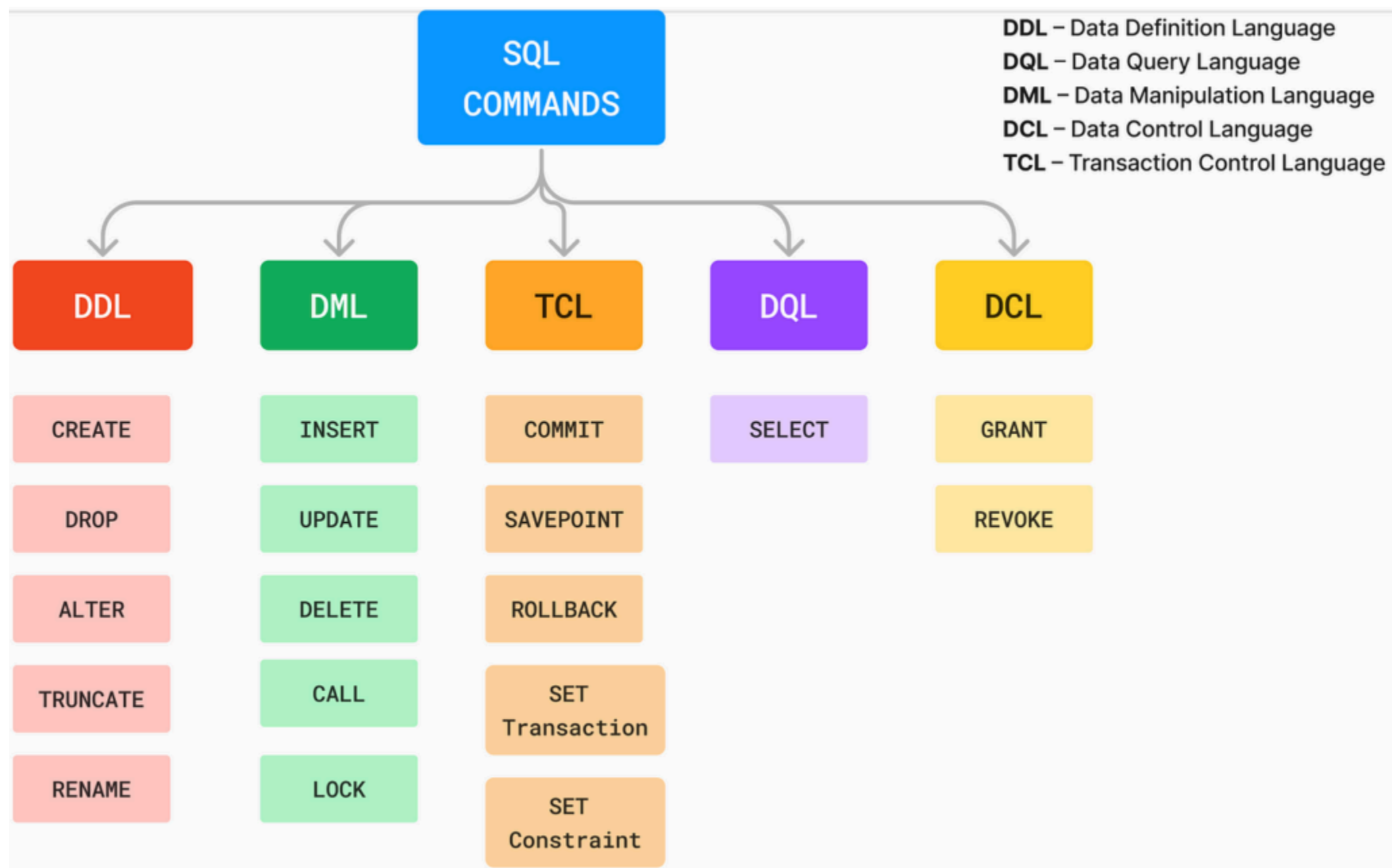
5. TCL (Transaction Control Language)

Used to **manage transactions** in a database.

- COMMIT → save changes permanently.
- ROLLBACK → undo changes.
- SAVEPOINT → set a point in a transaction to roll back to.

Quick Summary:

- **DDL** → Structure (CREATE, ALTER, DROP, TRUNCATE)
- **DML** → Data manipulation (INSERT, UPDATE, DELETE)
- **DQL** → Query (SELECT)
- **DCL** → Permissions (GRANT, REVOKE)
- **TCL** → Transactions (COMMIT, ROLLBACK, SAVEPOINT)



Step by step Installation process

❓ Install SQL server

<https://www.microsoft.com/en-in/sql-server/sql-server-downloads>

❓ Install SSMS

[Install SQL Server Management Studio | Microsoft Learn](#)

❓ Create The database

SQL Server Management Studio

SSMS stands for **SQL Server Management Studio**. It's the **official graphical tool** from **Microsoft** used to manage SQL Server databases. Think of it as the **control center** for your SQL Server.

1. Key Features of SSMS

1. Query Editor

- Write and execute **T-SQL queries**.
- Supports syntax highlighting, IntelliSense, and query execution plans.

2. Object Explorer

- Browse all **databases, tables, views, stored procedures, and security settings**.
- Allows you to create, modify, or delete objects.

3. Management Tools

- Backup and restore databases.
- Configure users, roles, and permissions.
- Set up **SQL Server Agent jobs** (automation).

4. Performance Monitoring

- Analyze query performance using execution plans.
- Monitor CPU, memory, and I/O usage.

5. Integration

- Works with **SQL Server, Azure SQL, and SQL Server on Docker or Linux**.
- Can integrate with **Visual Studio** and other tools.

--- sample sql query ===== SQL statement

SELECT ===== Clauses SELECT/FROM/WHERE

name,

LOWER(state) ===== all green keywords

FROM customers telanagana=value = operator

WHERE country = 'Telangana'; all Black colors identifier

Statements, Clauses, and Keywords in SQL

1. SQL Statement

- A **complete instruction** that SQL understands and executes.
- Ends with a semicolon (;).
- Made up of clauses.

Example:

```
SELECT name, salary  
FROM employees  
WHERE salary > 50000;
```

This entire block is **one SQL statement**.

2. SQL Clause

- A **part of a statement**.
- Each clause performs a specific role like filtering, grouping, or ordering.
- Example clauses: [SELECT](#), [FROM](#), [WHERE](#), [GROUP BY](#), [HAVING](#), [ORDER BY](#).

Example:

```
WHERE salary > 50000
```

is the **WHERE clause** of the above statement.

3. SQL Keyword

- Reserved **words with special meaning** in SQL.
- They form the building blocks of statements and clauses.
- Examples: [SELECT](#), [FROM](#), [WHERE](#), [INSERT](#), [UPDATE](#), [DELETE](#), [JOIN](#), [ORDER BY](#), [DISTINCT](#).
- Keywords are **case-insensitive** (SELECT = select).

Example:

In

```
SELECT name FROM employees;
```

- SELECT and FROM are **keywords**.
- Together, they form **clauses**.
- The whole thing is a **statement**.

Putting It All Together

Think of it like English grammar:

- **Statement** = full sentence
- **Clause** = part of a sentence (subject, predicate, etc.)
- **Keyword** = grammar words (like "and", "if", "then")
-

USE database

[?] -- comments single line comments

```

/* multi line comments
Hi
All
*/

```

❓ SELECT and FROM

```
SELECT *
FROM customers
```

* === (all)

--Specific orders from column

```
SELECT
    first_name,
    country,
    score
FROM dbo.customers
```

```
/* Specific orders from column can be interchanged result will change according to that */
```

```
SELECT
    first_name,
    score
    country
FROM dbo.customers
```

WHERE clause

/*Where is based on condition like score >500, score=0, or country=usa*/

--Specific orders from column

SELECT

first_name,
score
country

FROM dbo.customers

WHERE score <500 -- change conditions

--Specific orders from column

SELECT

first_name,
score
country

FROM dbo.customers

WHERE country = 'germany' # string value must be in ''

ORDER BY Clause

/*Order your data as per asc or dec order

Asc=lower to Higher

Desc= higher to lower

Kindly note if we are not defining any format by default format is asc*/

SELECT *

```
FROM customers
ORDER BY score
```

/*Order your data as per asc or dec order
Asc=lower to Higher
Dec= higher to lower

Kindly note if we are not defining any format by default format is asc*/

```
SELECT *
FROM customers
ORDER BY score desc
```

```
/* nested sorting */
SELECT *
FROM customers
ORDER BY
    country asc,
    score desc
```

GROUP BY clause

What is GROUP BY?

- The GROUP BY clause **groups rows that have the same values in one or more columns.**
- Often used with **aggregate functions** like **COUNT(), SUM(), AVG(), MAX(), MIN().**
- It helps summarize data instead of showing every row individually

```
/* will sum Up two country and add score */
SELECT
    country,
    SUM(score)
```

```
FROM customers
GROUP BY country
```

```
/* will sum Up two country and add score */
SELECT
    country,
```

```
    SUM(score) AS total_sum  -- 1. Column Alias Rename a column in the result set
```

```
FROM customers
GROUP BY country
```

```
/* will sum Up two country and add score */
```

```
SELECT
    country,
    avg(score) AS total_sum,  -- 1. Column Alias Rename a column in the result set
```

```
    COUNT (id) as total_cus
FROM customers
GROUP BY country
```

HAVING Clause also called filtering

```
/* will sum Up two country and add score and filter sum of the score above 400 */
```

```
SELECT
    country,
    SUM(score) AS total_sum  -- 1. Column Alias Rename a column in the result set
```

```
FROM customers
GROUP BY country
HAVING SUM(score)>800
```

/*Difference between WHERE and HAVING*/

Filters **rows before any grouping** happens.

Filters **rows after any grouping** happens.

```
/* find avg score of each country
add customers score not equal to zero
```

and return only those country whose score is greater than 500
*/
SELECT
 country,
 sum(score) AS sum_score
FROM customers
WHERE score!=0
GROUP BY Country
HAVING sum(score)>500

Day 03

TOP

In **SQL**, the **TOP clause** is used to **limit the number of rows** returned by a query.

-- list top 2 value of countries

use mydatabase

```
SELECT TOP 2 *  
FROM customers
```

-- list top 4 value of countries as per lowest score

use mydatabase

```
SELECT TOP 4 *  
FROM customers  
ORDER BY score asc
```

General SQL Query Writing Order

In SQL, the **order of coding (query writing)** follows a **logical sequence**, but the way you **write** a query is different from the way SQL actually **executes** it.

When you write a query, you usually follow this order:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition  
GROUP BY column  
HAVING condition  
ORDER BY column
```

Actual SQL Execution Order

SQL doesn't execute in the same order you write.

The database engine processes queries in this **logical order**:

1. **FROM** → Choose the table(s).
2. **JOIN** → Apply joins (if any).
3. **WHERE** → Filter rows.
4. **GROUP BY** → Group rows.
5. **HAVING** → Filter groups.
6. **SELECT** → Pick columns or expressions.
7. **DISTINCT** → Remove duplicates.
8. **ORDER BY** → Sort the result.
9. **TOP / LIMIT** → Return only limited rows.

Example format

```
SELECT  
    country,  
    sum(score) AS sum_score  
FROM customers  
WHERE score!=0  
GROUP BY Country  
HAVING sum(score) >500  
ORDER BY country
```

```
/* to run query from multiple tables */  
SELECT *
```

FROM customers; -- ; is optional

SELECT *
FROM orders

Data types in SQL

Different data types in SQL. SQL supports several categories of data types, mainly numeric, string (character), date/time, and others.

1 Numeric Data Types

Used to store numbers.

Type	Description	Example Values
INT / INTEGER	Whole numbers (positive/negative)	1, -25, 1000
SMALLINT	Smaller whole numbers	100, -300
BIGINT	Very large whole numbers	100000000000
DECIMAL(p,s) / NUMERIC(p,s)	Fixed-point numbers with precision p and scale s	712345.6
FLOAT / REAL	Approximate floating-point numbers	3.14159, 0.001
BIT / BOOLEAN	True/False values (1/0)	0, 1

2 Character / String Data Types

Used to store text.

Type	Description	Example Values
CHAR(n)	Fixed-length string of n characters	'John ' (padded)
VARCHAR(n)	Variable-length string, max n characters	'John', 'Alice'
TEXT / CLOB	Very large text (depends on DBMS)	Long paragraphs

3 Date / Time Data Types

Used to store dates and times.

Type	Description	Example Values
DATE	Stores date only (YYYY-MM-DD)	2025-10-08
TIME	Stores time only (HH:MM:SS)	14:30:00
DATETIME / TIMESTAMP	Stores date and time together	2025-10-08 14:30:00
YEAR (MySQL)	Stores year only	2025

4 Other / Special Data Types

Type	Description	Example
BOOLEAN	True/False	TRUE, FALSE
BINARY / VARBINARY	Store binary data (images, files)	0xFFD8FFE0
ENUM (MySQL)	List of allowed values	'Small', 'Medium', 'Large'
JSON	JSON formatted data (supported in MySQL/PostgreSQL)	'{"name":"John"}'
UUID / UNIQUEIDENTIFIER	Globally unique identifier	'550e8400-e29b-41d4-a716-446655440000'

Quick Tips

- **Choose the smallest data type that fits your data** → saves storage.
- **Use numeric types** for calculations, **string types** for text, **date/time types** for timestamps.
- **Constraints** like NOT NULL, PRIMARY KEY, or DEFAULT often go together with data types.

DDL (data definition language)

1. **CREATE DATABASE** → Makes SampleDB.
2. **USE** → Switch into that database.
3. **CREATE TABLE** → Makes Employees with 3 columns.
4. **ALTER** → Adds Department, changes Salary precision.
5. **TRUNCATE** → Clears all rows but keeps the table.
6. **DROP TABLE** → Deletes the Employees table.
7. **DROP DATABASE** → Deletes the whole SampleDB.

-- 1. Create a new database
CREATE DATABASE SampleDB;

-- 3. Create a new table
CREATE TABLE Employees (
 EmpID INT PRIMARY KEY, -- must be unique can't be null

```
    Name VARCHAR(100),  
    Salary DECIMAL(10,2)  
);
```

```
SELECT *  
FROM Employees
```

-- 4. Alter table: Add a new column

```
ALTER TABLE Employees ADD Department VARCHAR(50);
```

-- 5. Alter table: Modify column data type

```
ALTER TABLE Employees ALTER COLUMN Salary DECIMAL(12,2);
```

-- 6. Truncate table: Remove all rows (structure stays)

```
TRUNCATE TABLE Employees;
```

-- 7. Drop the table

```
DROP TABLE Employees;
```

-- 8. Drop the database

```
DROP DATABASE SampleDB;
```

What DML (data manipulation language)

DML (Data Manipulation Language)

- DML commands are used to **work with the data inside tables** (not the structure).
- They let you **insert, update, delete, and read** records.
- DML statements can be **rolled back** (undone) if used inside a transaction.

DML and DDL flow

-- Create a sample database

```
CREATE DATABASE DMLDemoDB;  
USE DMLDemoDB;
```

-- Create Employees table

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Salary DECIMAL(10,2),  
    Department VARCHAR(50)
```

```
);
```

```
-- insert data into table
```

```
INSERT INTO Employees (EmpID, Name, Salary, Department)
VALUES (1, 'John Doe', 50000, 'HR');
```

```
INSERT INTO Employees (EmpID, Name, Salary, Department)
VALUES (2, 'Alice Smith', 60000, 'IT'),
       (3, 'Bob Johnson', 55000, 'Finance');
```

```
-- check for updates
```

```
SELECT * FROM Employees;
```

```
--update table data
```

```
UPDATE Employees
SET Salary = 65000
WHERE EmpID = 2;
```

```
-- delete data
```

```
DELETE FROM Employees
WHERE EmpID = 3;
```

1. Basic Filtering Using WHERE

The WHERE clause is used to **filter rows based on a condition**.

```
SELECT *
FROM Employees
WHERE Department = 'IT';
```

Retrieves only employees in the IT department.

2. Using Comparison Operators

Operator	Meaning	Example
=	Equal to	Salary = 50000
<> or !=	Not equal to	Department <> 'HR'
>	Greater than	Salary > 40000
>=	Greater than or equal to	Salary >= 50000
<	Less than	Salary < 60000
<=	Less than or equal to	Salary <= 55000

```
SELECT Name, Salary
FROM Employees
WHERE Salary > 50000;
```

3. Using Logical Operators

Operator	Meaning	Example
AND	Both conditions must be true	Salary > 50000 AND Department='IT'
OR	Either condition is true	Department='IT' OR Department='HR'
NOT	Negates a condition	NOT Department='Finance'

```
SELECT *
FROM Employees
WHERE Salary > 50000 AND Department = 'IT';
```

4. Filtering with LIKE (Pattern Matching)

- % → any number of characters
- _ → single character

```
-- Names starting with 'A'
SELECT *
FROM Employees
WHERE Name LIKE 'A%';
```

```
-- Names with second letter 'o'
SELECT *
FROM Employees
WHERE Name LIKE '___o%';
```

5. Filtering with IN / NOT IN

```
-- Employees in IT or HR
SELECT *
FROM Employees
```

```
WHERE Department IN ('IT', 'HR');
```

```
-- Employees NOT in Finance
```

```
SELECT *  
FROM Employees  
WHERE Department NOT IN ('Finance');
```

6. Filtering with BETWEEN

```
-- Salary between 40000 and 60000
```

```
SELECT *  
FROM Employees  
WHERE Salary BETWEEN 40000 AND 60000;
```

7. Filtering with IS NULL / IS NOT NULL

```
-- Employees without a department
```

```
SELECT *  
FROM Employees  
WHERE Department IS NULL;
```

```
-- Employees with a department
```

```
SELECT *  
FROM Employees  
WHERE Department IS NOT NULL;
```

We'll make a database CompanyDB with two main tables:

- Employees
 - Departments
-

Step 1: Create Database

```
CREATE DATABASE CompanyDB;  
USE CompanyDB;
```

Step 2: Create Tables

```
-- Departments Table
```

```
CREATE TABLE Departments (  
    DepartmentID INT PRIMARY KEY,
```

```

        DepartmentName VARCHAR(100)
    );

-- Employees Table
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(100),
    DepartmentID INT,
    ManagerID INT NULL,

-- for SELF JOIN demo
    FOREIGN KEY (DepartmentID) REFERENCES
Departments(DepartmentID)
);

```

Step 3: Insert Sample Data

```

-- Insert Departments
INSERT INTO Departments (DepartmentID, DepartmentName) VALUES
(1, 'HR'),
(2, 'Finance'),
(3, 'IT'),
(4, 'Sales');

-- Insert Employees
INSERT INTO Employees (EmpID, Name, DepartmentID, ManagerID)
VALUES
(101, 'Alice', 1, NULL),           -- HR, no manager
(102, 'Bob', 2, 101),             -- Finance, managed by Alice
(103, 'Charlie', 3, 101),         -- IT, managed by Alice
(104, 'David', NULL, 102),        -- No department, managed by Bob
(105, 'Eva', 4, 103),            -- Sales, managed by Charlie
(106, 'Frank', 2, NULL);         -- Finance, no manager

```

Now you have:

- 4 departments
- 6 employees (some have managers, some not; some belong to no department)

SQL Joins:

A **JOIN** in SQL is used to combine rows from two or more tables based on a related column between them.

Returns only the rows that have matching values in both tables.

Inner Join

```
SELECT e.EmpID, e.Name, d.DepartmentName
FROM Employees e
INNER JOIN Departments d
ON e.DepartmentID = d.DepartmentID;
```

Only employees that belong to an existing department.

LEFT JOIN (or LEFT OUTER JOIN)

Returns **all rows from the left table**, and matched rows from the right table.

If no match → NULL on right side.

```
SELECT e.EmpID, e.Name, d.DepartmentName
FROM Employees e
LEFT JOIN Departments d
ON e.DepartmentID = d.DepartmentID;
```

Shows all employees, even if some don't belong to any department.

RIGHT JOIN (or RIGHT OUTER JOIN)

Returns **all rows from the right table**, and matched rows from the left table.

If no match → NULL on left side.

```
SELECT e.EmpID, e.Name, d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d
ON e.DepartmentID = d.DepartmentID;
```

Shows all departments, even if no employee belongs to them.

FULL JOIN (or FULL OUTER JOIN)

Returns all rows when there is a match in **either** left or right table.

Non-matching rows get NULL.

```
SELECT e.EmpID, e.Name, d.DepartmentName
FROM Employees e
FULL JOIN Departments d
```

`ON e.DepartmentID = d.DepartmentID;`

Combines all employees + all departments, even if no match.

CROSS JOIN

Returns **Cartesian product** (every row of first table with every row of second).

No ON condition needed.

```
SELECT e.Name, d.DepartmentName
FROM Employees e
      CROSS JOIN Departments d;
```

If 5 employees × 3 departments → 15 rows.

SELF JOIN

-- Table joins with itself.

```
SELECT e1.Name AS Employee, e2.Name AS Manager
FROM Employees e1
      INNER JOIN Employees e2
      ON e1.ManagerID = e2.EmpID;
```

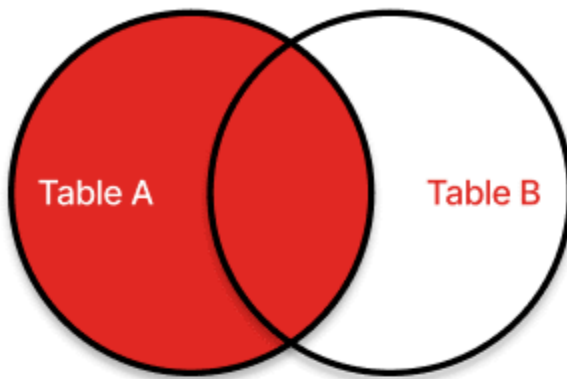
Used for hierarchical data (employee-manager relationships).

Visual Summary:

Join Type	Rows Returned
INNER JOIN	Matches only
LEFT JOIN	All left + matches
RIGHT JOIN	All right + matches
FULL JOIN	All rows from both
CROSS JOIN	Cartesian product
SELF JOIN	Same table join

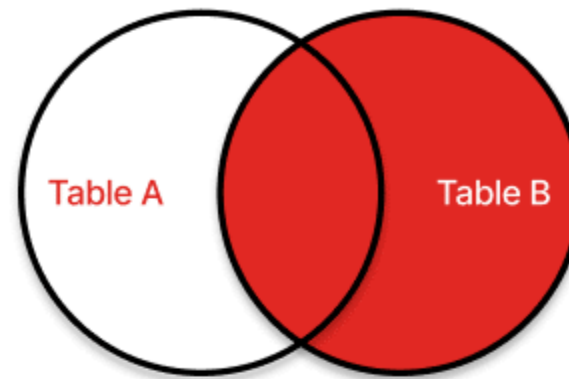
SQL JOINS Visualized with Venn Diagrams

LEFT JOIN



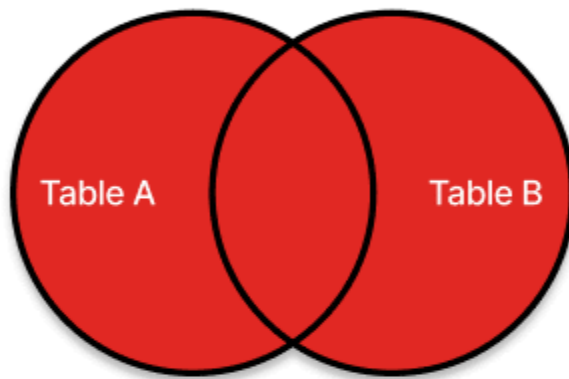
```
SELECT * FROM A LEFT JOIN B ON A.key = B.key;
```

RIGHT JOIN



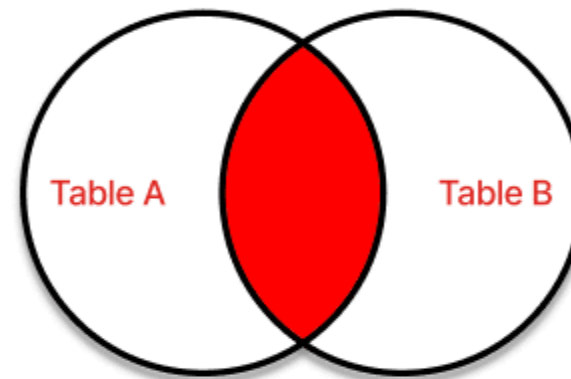
```
SELECT * FROM A RIGHT JOIN B ON A.key = B.key;
```

FULL OUTER JOIN



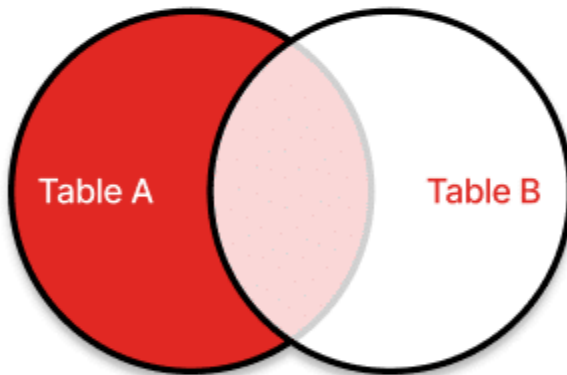
```
SELECT * FROM A FULL OUTER JOIN B ON A.key = B.key;
```

INNER JOIN



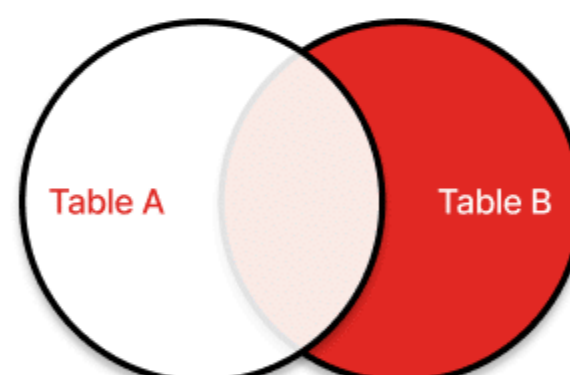
```
SELECT * FROM A INNER JOIN B ON A.key = B.key;
```

LEFT JOIN excluding INNER JOIN



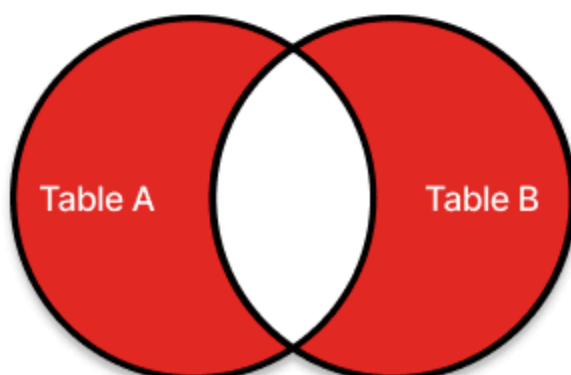
```
SELECT * FROM A LEFT JOIN B ON A.key = B.key  
WHERE B.key IS NULL;
```

RIGHT JOIN excluding INNER JOIN



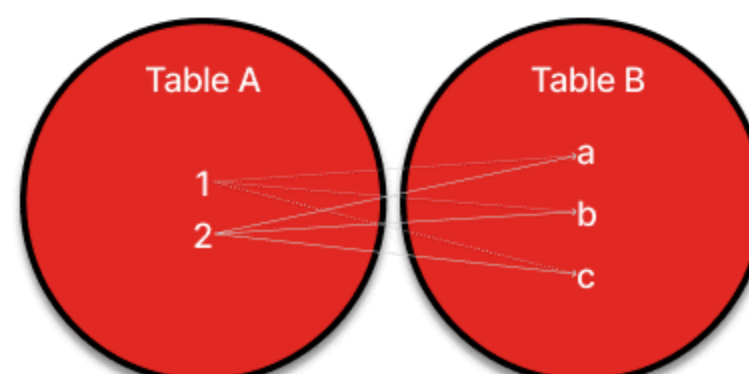
```
SELECT * FROM A RIGHT JOIN B ON A.key = B.key  
WHERE A.key IS NULL;
```

FULL OUTER JOIN excluding INNER JOIN



```
SELECT * FROM A FULL OUTER JOIN B ON A.key = B.key  
WHERE A.key IS NULL OR B.key IS NULL;
```

CROSS JOIN



```
SELECT * FROM A CROSS JOIN B;
```

SQL Set Operators

. These operate **on the results of two queries**, similar to how joins combine tables, but they combine **query results as sets of rows**.

1. UNION

Combines results of **two queries** and **removes duplicates** by default.

Columns must match in **number and datatype**.

```
-- Employees from HR or Finance (just names)
SELECT Name FROM Employees WHERE DepartmentID = 1
UNION
SELECT Name FROM Employees WHERE DepartmentID = 2;
```

Result: Alice, Bob, Frank

- Alice → HR
- Bob, Frank → Finance
- Duplicates automatically removed.

2. UNION ALL

Combines results of **two queries**, **keeps duplicates**.

```
SELECT Name FROM Employees WHERE DepartmentID = 1
UNION ALL
SELECT Name FROM Employees WHERE DepartmentID = 2;
```

Result: Alice, Bob, Frank (duplicates would appear if present)

3. INTERSECT

```
-- Names in both HR and Finance (none here)
SELECT Name FROM Employees WHERE DepartmentID = 1
INTERSECT
SELECT Name FROM Employees WHERE DepartmentID = 2;
```

Returns **only rows common to both queries**.

Result: *Empty set* (no employee is in both HR and Finance)

4. EXCEPT (or MINUS in some DBs)

Returns rows from the **first query** that are **not in the second**.

```
-- Employees in HR but NOT in Finance
SELECT Name FROM Employees WHERE DepartmentID = 1
EXCEPT
```

`SELECT Name FROM Employees WHERE DepartmentID = 2;`
Result: Alice

Quick Visual Summary

Operator	Meaning	Duplicates	Example Result
UNION	Combine & remove duplicates	No	Alice, Bob, Frank
UNION ALL	Combine & keep duplicates	Yes	Alice, Bob, Frank
INTERSECT	Only rows present in both queries	No	(empty)
EXCEPT / MINUS	Rows in first query but not in second	No	Alice

What are SQL Functions ?

SQL functions are built-in routines that perform operations on data and return results. They help manipulate data, perform calculations, and transform values within SQL queries.

Types of SQL Functions

1. Scalar Functions

Return a single value based on input value(s).

String Functions

LEN() or LENGTH() Function

It returns the **number of characters** in a string (excluding trailing spaces).

-- LENGTH / LEN

`SELECT name, LEN(name) AS name_length FROM employees;`

UPPER / LOWER

Overview

They are used to **convert the case** of text values:

- UPPER() → Converts text to **uppercase**
- LOWER() → Converts text to **lowercase**

```
-- UPPER / LOWER
```

```
SELECT UPPER(name) AS upper_name, LOWER(name) AS lower_name  
FROM employees;
```

Overview: What is CONCAT()

The **CONCAT()** function joins (combines) **two or more strings** into a single string. It's commonly used to create **full names**, **addresses**, or **messages** from multiple columns.

```
-- CONCAT
```

```
SELECT CONCAT(name, ' ', name) AS full_name FROM employees;
```

Overview: What is TRIM()?

The **TRIM()** function is used to **remove unwanted spaces or specific characters** from the **beginning and/or end** of a string.

It's commonly used for **data cleaning** — for example, when data imported from Excel or user input has extra spaces.

```
-- TRIM
```

```
SELECT TRIM(' Hello World ') AS trimmed_string;
```

REPLACE() function in SQL

— it's very handy for **modifying text values**.

Overview: What is REPLACE()?

The **REPLACE()** function **replaces all occurrences of a substring** in a string with a new substring.

- Very useful for **data cleaning**, correcting typos, or standardizing text.

- Works on **columns or string literals**.

Syntax

REPLACE(original_string, old_substring, new_substring)

- **original_string** → The string you want to modify
- **old_substring** → Text to find
- **new_substring** → Text to replace it with

Example 1 — Simple Replacement

```
SELECT REPLACE('Hello World', 'World', 'SQL') AS UpdatedString;
```

Output:

UpdatedString

Hello SQL

-- REPLACE

```
SELECT REPLACE(department, 'Engineering', 'Tech') AS  
updated_dept FROM employees;
```

REVERSE() function in SQL

— it's simple but useful for string manipulation.

Overview: What is REVERSE()

The **REVERSE()** function **reverses the order of characters** in a string.

- Useful for **text transformations**, checking palindromes, or certain data-cleaning tasks.
- Works on **columns or string literals**.

Syntax

REVERSE(string_expression)

- **string_expression** → The string you want to reverse

-- REVERSE

```
SELECT REVERSE(firstname) AS reversed_name FROM employees;
```

sample database with an **Employees table** including **date and time columns**, insert some data, and then show **examples of date/time functions** on that data.

1. Create Database and Table

-- Create Database

```
CREATE DATABASE CompanyDB;
```

-- Use the Database

```
USE CompanyDB;
```

-- Create Employees Table

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    Department VARCHAR(50),  
    HireDate DATE,  
    BirthDate DATE,  
    Salary DECIMAL(10,2)  
);
```

Insert Sample Data

```
INSERT INTO Employees (EmpID, FirstName, LastName, Department,  
HireDate, BirthDate, Salary)
```

```
VALUES
```

```
(1, 'Pawan', 'Kumar', 'Engineering', '2019-05-10',  
'1990-12-07', 90000),
```

```
(2, 'Neha', 'Sharma', 'Marketing', '2021-11-23', '1992-06-15',  
75000),
```

```
(3, 'Amit', 'Singh', 'Finance', '2018-03-15', '1988-09-20',  
95000),
```

```
(4, 'Riya', 'Verma', 'Engineering', '2020-01-10', '1995-04-05',  
80000),
```

```
(5, 'Rahul', 'Patel', 'HR', '2022-07-01', '1991-08-12', 70000);
```

We now have 5 employees with HireDate and BirthDate for testing date functions.

3. Examples of Date/Time Functions

3.1 Current Date and Time

```
SELECT GETDATE() AS CurrentDateTime, CURRENT_TIMESTAMP AS  
CurrentTimeStamp;
```

3.2 Years Worked (DATEDIFF)

```
SELECT  
    FirstName,  
    HireDate,  
    DATEDIFF(YEAR, HireDate, GETDATE()) AS YearsWorked  
FROM Employees;
```

3.3 Hire Year, Month, Day (DATEPART)

```
SELECT  
    FirstName,  
    DATEPART(YEAR, HireDate) AS HireYear,  
    DATEPART(MONTH, HireDate) AS HireMonth,  
    DATEPART(DAY, HireDate) AS HireDay  
FROM Employees;
```

3.4 Probation End and Anniversary (DATEADD)

```
SELECT  
    FirstName,  
    HireDate,  
    DATEADD(MONTH, 6, HireDate) AS ProbationEnd,  
    DATEADD(YEAR, 1, HireDate) AS FirstAnniversary  
FROM Employees;
```

3.5 Format Date (FORMAT)

```
SELECT  
    FirstName,  
    HireDate,  
    FORMAT(HireDate, 'yyyy-MM-dd') AS ShortDate,  
    FORMAT(HireDate, 'MMMM dd, yyyy') AS LongDate  
FROM Employees;
```

3.6 End of Month (EOMONTH)

```
SELECT  
    FirstName,  
    HireDate,  
    EOMONTH(HireDate) AS EndOfMonth
```

FROM Employees;

3.7 Birthdays This Month

```
SELECT
    FirstName,
    BirthDate
FROM Employees
WHERE MONTH(BirthDate) = MONTH(GETDATE());
```

This returns employees whose birthday is in the **current month**.

4. Sample Output (Some Examples)

FirstName	HireDate	YearsWorked	ProbationEnd	FirstAnniversary	EndOfMonth
Pawan	2019-05-10	6	2019-11-10	2020-05-10	2019-05-31
Neha	2021-11-23	4	2022-05-23	2022-11-23	2021-11-30
Amit	2018-03-15	7	2018-09-15	2019-03-15	2018-03-31

This setup gives you a **working database** with data to practice **all major date/time functions** (GETDATE, DATEPART, DATEDIFF, DATEADD, FORMAT, EOMONTH) in SQL Server.

combine **string functions** and **date/time functions** into some **high-end, practical SQL examples**. These are great for real-world reporting and analytics.

Sample Database Setup

```
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Department VARCHAR(50),
    HireDate DATE,
    BirthDate DATE,
    Salary DECIMAL(10,2)
);

INSERT INTO Employees (EmpID, FirstName, LastName, Department,
HireDate, BirthDate, Salary)
```

VALUES

```
(1, 'Pawan', 'Kumar', 'Engineering', '2019-05-10',  
'1990-12-07', 90000),  
(2, 'Neha', 'Sharma', 'Marketing', '2021-11-23', '1992-06-15',  
75000),  
(3, 'Amit', 'Singh', 'Finance', '2018-03-15', '1988-09-20',  
95000),  
(4, 'Riya', 'Verma', 'Engineering', '2020-01-10', '1995-04-05',  
80000),  
(5, 'Rahul', 'Patel', 'HR', '2022-07-01', '1991-08-12', 70000);
```

High-End Example 1 — Employee Full Info Report

--- Combine CONCAT, UPPER, FORMAT, DATEDIFF:

SELECT

```
    EmpID,  
    CONCAT(UPPER(FirstName), ' ', UPPER(LastName)) AS FullName,  
    Department,  
    FORMAT(HireDate, 'MMMM dd, yyyy') AS HireDateFormatted,  
    FORMAT(BirthDate, 'dd-MM-yyyy') AS BirthDateFormatted,  
    DATEDIFF(YEAR, HireDate, GETDATE()) AS YearsWorked,  
    CASE  
        WHEN DATEDIFF(YEAR, HireDate, GETDATE()) >= 5 THEN  
'Senior'  
        ELSE 'Junior'  
    END AS ExperienceLevel  
FROM Employees  
ORDER BY YearsWorked DESC;
```

What it does:

- Converts full name to uppercase
- Formats dates for readability
- Calculates experience in years
- Categorizes as Senior/Junior

High-End Example 2 — Upcoming Birthdays

-- Combine **MONTH, DAY, CONCAT, FORMAT**:

```
SELECT
    CONCAT(FirstName, ' ', LastName) AS FullName,
    BirthDate,
    FORMAT(BirthDate, 'MMMM dd') AS BirthDayMonth
FROM Employees
WHERE MONTH(BirthDate) = MONTH(GETDATE())
ORDER BY DAY(BirthDate);
```

What it does:

- Finds employees whose birthday is in the **current month**
- Displays nicely formatted date
- Full name in one column

High-End Example 3 — Probation & Anniversary Report

-- Combine **DATEADD, DATEDIFF, CONCAT, FORMAT**:

```
SELECT
    CONCAT(FirstName, ' ', LastName) AS EmployeeName,
    Department,
    HireDate,
    DATEADD(MONTH, 6, HireDate) AS ProbationEndDate,
    DATEADD(YEAR, 1, HireDate) AS FirstAnniversary,
    DATEDIFF(DAY, GETDATE(), DATEADD(MONTH, 6, HireDate)) AS
DaysUntilProbationEnd
FROM Employees
ORDER BY DaysUntilProbationEnd ASC;
```

What it does:

- Shows **probation end date** and **first anniversary**
- Calculates **days left until probation ends**
- Helps HR schedule reminders

High-End Example 4 — Salary and Name Analytics

--Combine **LEN, UPPER, CONCAT, DATEDIFF, ROUND**:

```
SELECT
    CONCAT(UPPER(FirstName), ' ', UPPER(LastName)) AS
EmployeeName,
    LEN(FirstName) + LEN(LastName) AS NameLength,
    Salary,
```

```

        ROUND(Salary * 0.10, 2) AS Bonus10Percent,
        DATEDIFF(YEAR, HireDate, GETDATE()) AS YearsWorked,
        ROUND(Salary / DATEDIFF(YEAR, HireDate, GETDATE()), 2) AS
AvgAnnualEarnings
FROM Employees
WHERE Department = 'Engineering';

```

What it does:

- Calculates **name length**
- Computes **10% bonus**
- Calculates **average annual earnings** based on tenure

High-End Example 5 — Full Report with Conditional Formatting

--Combine **CONCAT**, **UPPER**, **LOWER**, **FORMAT**, **DATEDIFF**, **CASE**:

```

SELECT
    EmpID,
    CONCAT(UPPER(LEFT(FirstName, 1)),
    LOWER(SUBSTRING(FirstName, 2, LEN(FirstName)))) AS
ProperFirstName,
    CONCAT(UPPER(LEFT(LastName, 1)),
    LOWER(SUBSTRING(LastName, 2, LEN(LastName)))) AS ProperLastName,
    Department,
    FORMAT(HireDate, 'dd/MM/yyyy') AS HireDate,
    DATEDIFF(YEAR, HireDate, GETDATE()) AS ExperienceYears,
    CASE
        WHEN DATEDIFF(YEAR, HireDate, GETDATE()) >= 5 THEN
'Senior'
        WHEN DATEDIFF(YEAR, HireDate, GETDATE()) >= 2 THEN
'Mid-level'
        ELSE 'Junior'
    END AS ExperienceLevel
FROM Employees
ORDER BY ExperienceYears DESC;

```

What it does:

- Properly capitalizes names
- Formats hire date
- Calculates experience
- Categorizes employees (Senior, Mid-level, Junior)

What is a NULL Function in SQL?

In SQL, a **NULL function** helps you handle **NULL values** — which represent **missing or unknown data** in a table.
NULL ≠ 0, NULL ≠ ' ', NULL ≠ any value — it simply means “no value”.

Common NULL Handling Functions

Function	Description	Example
ISNULL() (SQL Server)	Replaces NULL with a specified value	SELECT ISNULL(Bonus, 0) AS Bonus FROM Employees;
IFNULL() (MySQL)	Similar to ISNULL()	SELECT IFNULL(Bonus, 0) FROM Employees;
COALESCE()	Returns the first non-null value in a list	SELECT COALESCE(Bonus, Commission, 0) AS TotalIncome FROM Employees;
NULLIF()	Returns NULL if two values are equal	SELECT NULLIF(Salary, 0) AS SalaryValue FROM Employees;

Example: Using ISNULL() and COALESCE()

Create a sample table

```
CREATE DATABASE testDB

use testDB

CREATE TABLE Employees (
    EmpID INT,
    Name VARCHAR(50),
    Salary DECIMAL(10,2),
    Bonus DECIMAL(10,2)
);

INSERT INTO Employees VALUES
(1, 'Pawan', 60000, NULL),
(2, 'Neha', 55000, 5000),
```

```
(3, 'Amit', 70000, NULL);
```

```
SELECT *  
FROM employees
```

Handle NULL values with ISNULL()

```
SELECT  
    EmpID,  
    Name,  
    Salary,  
    ISNULL(Bonus, 0) AS Bonus_Replaced  
FROM Employees;
```

Output:

Name	Salary	Bonus_New	TotalPay
Pawan	60000	0	60000
Neha	55000	5000	60000
Amit	70000	0	70000

Using COALESCE() (works in all SQL databases)

```
SELECT  
    EmpID,  
    Name,  
    Salary,  
    COALESCE(Bonus, 1000) AS Bonus_Replaced  
FROM Employees;
```

Using NULLIF()

```
SELECT
    Name,
    NULLIF(Salary, 70000) AS SalaryValue
FROM Employees;
```

Summary

Function	Use
ISNULL()	Replace NULL with a value (SQL Server)
IFNULL()	Replace NULL with a value (MySQL)
COALESCE()	Return first non-NULL value
NVL()	Replace NULL (Oracle)
NULLIF()	Return NULL if two values are equal

Step 4: Create a User-Defined NULL Function

You can also define your own function that handles nulls:

```
CREATE FUNCTION dbo.ReplaceNullBonus (@Bonus DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN ISNULL(@Bonus, 0)
END;
GO
```

```
-- Use it
SELECT
    EmpID,
    Name,
    Salary,
    dbo.ReplaceNullBonus(Bonus) AS Bonus_Replaced
FROM Employees;
```

ISNULL vs COALESCE

Feature	ISNULL()	COALESCE()
Handles multiple NULLs	✗ No	✓ Yes
SQL Standard	✗ SQL Server only	✓ ANSI Standard
Data Type Resolution	Based on first expression	Based on highest precedence
Performance	Slightly faster	Slightly slower
Use Case	Simple replacement	Complex or multi-check null handling

CASE WHEN statement — one of the most useful conditional tools in SQL.

What is CASE WHEN in SQL?

The **CASE** statement is like an **IF-THEN-ELSE** structure in programming. It allows you to apply conditions inside your SQL query and return specific values based on those conditions.

Basic Syntax

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default_result
END
The ELSE part is optional (if omitted, unmatched rows return NULL).
```

Use same DATABASE

Empl D	Name	Salary	Bonu s
1	Pawa n	6000 0	NULL
2	Neha	5500 0	5000

3	Amit	70000	NULL
---	------	-------	------

Classify employees based on salary.

```
SELECT
    Name,
    Salary,
    CASE
        WHEN Salary > 65000 THEN 'High Salary'
        WHEN Salary BETWEEN 55000 AND 65000 THEN 'Medium
Salary'
        ELSE 'Low Salary'
    END AS Salary_Category
FROM Employees;
```

Output:

Name	Salary	Salary_Category
Pawan	60000	Medium Salary
Neha	55000	Medium Salary
Amit	70000	High Salary

Example 2: Handling NULL values

You can use CASE to handle or replace NULL values (like ISNULL ()).

```
SELECT
    Name,
    CASE
        WHEN Bonus IS NULL THEN 0
        ELSE Bonus
    END AS Bonus_Fixed
FROM Employees;
```

Output:

Name	Bonus_Fixed
Pawan	0
Neha	5000
Amit	0

Example 3: Derived / Calculated Columns

You can compute **Total Compensation** including bonus — handling NULLs safely.

```
SELECT
    Name,
    Salary,
    Bonus,
    (Salary +
        CASE
            WHEN Bonus IS NULL THEN 0
            ELSE Bonus
        END
    ) AS Total_Compensation
FROM Employees;
```

Output:

Name	Salary	Bonus	Total_Compensation
Pawan	60000	NULL	60000
Neha	55000	5000	60000
Amit	70000	NULL	70000

Example 4: Nested CASE

You can also nest **CASE** inside another **CASE**:

```
SELECT
    Name,
    Salary,
```

```
CASE
    WHEN Salary > 65000 THEN
        CASE
            WHEN Bonus IS NULL THEN 'High Salary, No Bonus'
            ELSE 'High Salary, Has Bonus'
        END
    ELSE 'Not High Salary'
END AS Employee_Status
FROM Employees
```

Summary Table

Feature	Description	Example
CASE WHEN	Adds conditional logic	CASE WHEN Salary > 60000 THEN 'High' END
ELSE	Optional fallback	ELSE 'Unknown'
Used In	SELECT, WHERE, ORDER BY, GROUP BY	Anywhere you can use an expression
Similar To	IF-ELSE in other languages	—

Windows Functions

What is a Window Function?

A **window function** performs calculations **across a set of table rows that are somehow related** to the current row — **without grouping the data** like GROUP BY does. It looks at a **“window” (subset)** of rows and calculates things like:

- Running totals
- Ranking
- Averages
- Percentages
- Lag/Lead comparisons (previous/next row values)

Basic Syntax

```
function_name (expression)
OVER (
    [PARTITION BY column_name]
    [ORDER BY column_name]
)
```

Components:

- **function_name** → aggregate or analytic function (like SUM, AVG, ROW_NUMBER, etc.)
- **PARTITION BY** → divides data into groups (like GROUP BY)
- **ORDER BY** → defines order of rows inside each partition

Example Setup

```
CREATE TABLE Employees (
    EmpID INT,
    Name VARCHAR(50),
    Department VARCHAR(50),
    Salary DECIMAL(10,2)
);
```

```
INSERT INTO Employees VALUES
(1, 'Pawan', 'IT', 60000),
(2, 'Neha', 'IT', 70000),
(3, 'Amit', 'HR', 50000),
(4, 'Ravi', 'HR', 55000),
(5, 'Meena', 'Finance', 65000);
```

1. ROW_NUMBER()

Gives a unique sequential number to each row within a partition.

```
SELECT
    Name,
    Department,
```

```
Salary,
ROW_NUMBER() OVER (PARTITION BY Department ORDER BY Salary
DESC) AS RowNum
FROM Employees;
```

Output:

Name	Department	Salary	RowNum
Neha	IT	70000	1
Pawan	IT	60000	2
Ravi	HR	55000	1
Amit	HR	50000	2
Meena	Finance	65000	1

2. RANK() and DENSE_RANK()

Used for ranking rows.

```
SELECT
    Name,
    Department,
    Salary,
    RANK() OVER (ORDER BY Salary DESC) AS RankNo,
    DENSE_RANK() OVER (ORDER BY Salary DESC) AS DenseRankNo
FROM Employees;
```

Difference:

- RANK() leaves gaps if there are ties.
- DENSE_RANK() doesn't

Key Difference from GROUP BY

GROUP BY	WINDOW FUNCTION
----------	-----------------

Aggregates rows into one	Keeps all rows
You lose detail	You retain all rows
Used for summaries	Used for analytics

What are Aggregate Functions?

Aggregate functions perform a calculation on a **set of values** and return a **single value**.
They are commonly used with the GROUP BY clause but can also be used on the entire table.

Common Aggregate Functions in SQL

Function	Description	Example
SUM ()	Adds up numeric values	SUM (Salary)
AVG ()	Calculates average	AVG (Salary)
COUNT ()	Counts rows (or non-null values)	COUNT (EmpID)
MIN ()	Finds minimum value	MIN (Salary)
MAX ()	Finds maximum value	MAX (Salary)

Example Table: Employees

EmpID	Name	Salary	Bonus
1	Pawan	60000	NULL
2	Neha	55000	5000
3	Amit	70000	NULL

✓ 1. SUM()

Add up all salaries:
SELECT SUM(Salary) AS Total_Salary
FROM Employees;
Output:
Total_Salary
185000

✓ 2. AVG()

Average salary:

```
SELECT AVG(Salary) AS Average_Salary  
FROM Employees;
```

Output:

Average_Salary

61666.67

✓ 3. COUNT()

Count employees:

```
SELECT COUNT(EmpID) AS Total_Employees  
FROM Employees;
```

Output:

Total_Employees

3

Count non-null Bonus values:

```
SELECT COUNT(Bonus) AS Bonus_Exists  
FROM Employees;
```

Output:

Bonus_Exists

1

✓ 4. MIN() and MAX()

Minimum and maximum salaries:

```
SELECT MIN(Salary) AS Min_Salary,  
       MAX(Salary) AS Max_Salary  
FROM Employees;
```

Output:

Min_Salary Max_Salary

55000 70000

✓ 5. Using GROUP BY

Aggregate by department or category. Example if we add **Department**:

-- Example with department

```
SELECT Department, AVG(Salary) AS Avg_Salary  
FROM Employees  
GROUP BY Department;
```

Returns average salary per department instead of the whole table.

Key Points

1. Aggregate functions **ignore NULLs** (except COUNT(*) counts all rows).
2. Commonly combined with GROUP BY for grouped calculations.
3. Can also be used with HAVING to filter groups.

What are Window Functions?

A **window function** performs a calculation across a **set of rows related to the current row**, without collapsing rows into a single output (unlike regular aggregate functions).

- ✓ Key difference from normal aggregate functions:
- **Aggregate:** returns **one row per group** (SUM, AVG, COUNT with GROUP BY)
 - **Window:** returns a value for **each row**, computed over a **window of rows**

⚙ Syntax

```
AGG_FUNC() OVER (  
  [PARTITION BY column1, column2, ...]  
  [ORDER BY column3]  
)
```

- **PARTITION BY** → defines **groups** (like GROUP BY, but keeps individual rows)
- **ORDER BY** → defines the **order of rows** for running totals, ranks, etc.

Example Table: Employees

Empl D	Name	Salar y	Bonu s
1	Pawa n	60000	NULL
2	Neha	55000	5000
3	Amit	70000	NULL

✓ 1. Running Total (SUM)

Calculate running total of salaries ordered by salary:

Output:

```
SELECT
    Name,
    Salary,
    SUM(Salary) OVER (ORDER BY Salary) AS Running_Total
FROM Employees;
```

Name Salary Running_Total

Neha 55000 55000

Pawan 60000 115000

Amit 70000 185000

✓ 2. AVG over a Window

Average salary across all employees:

```
SELECT
    Name,
    Salary,
    AVG(Salary) OVER () AS Avg_Salary
FROM Employees;
```

Output:

Name	Salary	Avg_Salary
Pawan	60000	61666.67
Neha	55000	61666.67
Amit	70000	61666.67

Here, OVER() without PARTITION BY considers **all rows**.

3. PARTITION BY

Compute **average salary by department** (if we have a Department column):

```
SELECT
    Name,
    Department,
    Salary,
    AVG(Salary) OVER (PARTITION BY Department) AS
Dept_Avg_Salary
FROM Employees;
```

Each row shows **department-level average**, without collapsing rows.

✓ 4. ROW_NUMBER(), RANK(), DENSE_RANK()

Window functions are often used for **ranking**:

```
SELECT
```

```
Name ,
Salary ,
ROW_NUMBER() OVER (ORDER BY Salary DESC) AS Row_Num ,
RANK() OVER (ORDER BY Salary DESC) AS Rank_Num ,
DENSE_RANK() OVER (ORDER BY Salary DESC) AS Dense_Rank_Num
FROM Employees;
```

Output:

Name	Salary	Row_Num	Rank_Nu m	Dense_Rank_Num
Amit	70000	1	1	1
Pawan	60000	2	2	2
Neha	55000	3	3	3

◆ Key Points

1. Window functions **do not collapse rows** — unlike regular aggregates.
2. Must use OVER() clause.
3. Useful for **running totals, moving averages, ranking, lag/lead comparisons, cumulative sums**.